

MAHARISHI MARKANDESHWAR (DEEMED TO BE UNIVERSITY)

MM ENGINEERING COLLEGE (MMEC)

Mullana, Ambala, Haryana - 133207 • Accredited by NAAC with Grade 'A++'



OFFICIAL LABORATORY RECORD & MANUAL

Computational & Problem Solving using C Practical Manual

Course Code: BCSE-001 • Semester 1 & 2 (1st Year)

Department of Computer Science & Engineering • Session 2025-26

STUDENT PRACTICAL RECORD & EVALUATION DOSSIER

Student Name:

Roll Number:

University Reg No:

Class & Section:

Academic Program: CSE / IT / AI-DS / AI-ML / ECE

Curriculum Batch: 2025 - 2029 (Session 2025-26)

Signature of Laboratory Faculty Incharge

Signature of Head of Department

Exp #	Title / Aim of the Practical Experiment	Date of Perf.	Date of Subm.	Marks	Sign
1	Arithmetic Calculator with Modulo & Division by Zero Protection	/..../2025	/..../2025	/ 10	
2	Distance Converter (Kilometers to Meters, Feet, Inches & Centimeters)	/..../2025	/..../2025	/ 10	
3	Bitwise Operators Demonstration (&, , ^, ~, <<, >>)	/..../2025	/..../2025	/ 10	
4	Largest of Three Numbers using Conditional (Ternary) Operator	/..../2025	/..../2025	/ 10	
5	Leap Year Verification using Boolean Logic	/..../2025	/..../2025	/ 10	
6	Student Grade & Division Evaluator for 5 Subjects	/..../2025	/..../2025	/ 10	
7	Prefix vs Postfix Increment and Decrement Operator Analysis	/..../2025	/..../2025	/ 10	
8	Menu-Driven Switch Program (Even/Odd, Pos/Neg, Square, Square Root)	/..../2025	/..../2025	/ 10	
9	Sum of All Integers Between 100 and 200 Divisible by 5	/..../2025	/..../2025	/ 10	
10	Armstrong Numbers Discovery in Range [m, n]	/..../2025	/..../2025	/ 10	
11	Linear Search & Binary Search on 1D Array	/..../2025	/..../2025	/ 10	
12	Matrix Operations (Addition, Subtraction, Multiplication & Transpose)	/..../2025	/..../2025	/ 10	
13	Built-in String Library Functions Demonstration (string.h)	/..../2025	/..../2025	/ 10	
14	User-Defined Implementations of String Functions (Without string.h)	/..../2025	/..../2025	/ 10	
15	Call by Value vs Call by Reference (Swapping Two Variables)	/..../2025	/..../2025	/ 10	
16	Recursive Computation of Factorial	/..../2025	/..../2025	/ 10	
17	Array Reversal & Memory Address Inspection via Pointers	/..../2025	/..../2025	/ 10	
18	Function Returning Pointer to Larger of Two Integers	/..../2025	/..../2025	/ 10	
19	Structure personal (Name, Date of Joining, Salary) Database	/..../2025	/..../2025	/ 10	
20	Dynamic Memory Allocation (malloc, realloc & free) for Strings	/..../2025	/..../2025	/ 10	
21	File Handling: Store & Retrieve 15 Employee Records	/..../2025	/..../2025	/ 10	

Arithmetic Calculator with Modulo & Division by Zero Protection

🎯 AIM / OBJECTIVE:

Write a C program to perform basic arithmetic operations (Addition, Subtraction, Multiplication, Division, and Modulus) on two user input numbers with safe divide-by-zero check.

⚙️ ALGORITHM / STEP-BY-STEP PROCEDURE:

- Step 1: Start the program.
- Step 2: Declare two double variables num1, num2 and a char variable op.
- Step 3: Prompt user for operator (+, -, *, /, %) and two numbers.
- Step 4: Use switch(op) to branch calculations.
- Step 5: For division (/) and modulo (%), ensure num2 != 0 before computing.
- Step 6: Display calculated result with 2-place decimal precision.
- Step 7: Terminate program.

📄 SOURCE CODE IMPLEMENTATION (BCSE-001)

VERIFIED & TESTED

```
#include <stdio.h>

int main() {
    double num1, num2;
    char op;

    printf("*** BCSE-001: Practical 1 - Simple Calculator ***
");
    printf("Enter operator (+, -, *, /, %): ");
    scanf(" %c", &op);

    printf("Enter two numbers: ");
    scanf("%lf %lf", &num1, &num2);

    switch (op) {
        case '+':
            printf("Result: %.2lf + %.2lf = %.2lf
", num1, num2, num1 + num2);
            break;
        case '-':
            printf("Result: %.2lf - %.2lf = %.2lf
", num1, num2, num1 - num2);
            break;
        case '*':
            printf("Result: %.2lf * %.2lf = %.2lf
", num1, num2, num1 * num2);
            break;
        case '/':
            if (num2 != 0)
                printf("Result: %.2lf / %.2lf = %.4lf
", num1, num2, num1 / num2);
            else
                printf("Error: Division by zero is undefined in mathematics!
");
            break;
        case '%':
            if ((int)num2 != 0)
                printf("Result: %d %% %d = %d
", (int)num1, (int)num2, (int)num1 % (int)num2);
            else
                printf("Error: Modulo by zero is undefined!
");
            break;
        default:
            printf("Error: Invalid operator '%c' entered!
", op);
    }
}
```

```
return 0;  
}
```

**SAMPLE INPUT:**

Operator: /
Numbers: 25 4

**EXPECTED OUTPUT:**

Result: 25.00 / 4.00 = 6.2500

Distance Converter (Kilometers to Meters, Feet, Inches & Centimeters)

🎯 AIM / OBJECTIVE:

Write a C program to calculate and display the distance between two cities entered in Kilometers into Meters, Feet, Inches, and Centimeters.

⚙️ ALGORITHM / STEP-BY-STEP PROCEDURE:

- Step 1: Input distance in kilometers (km) as floating point.
- Step 2: Calculate meters = km * 1000.0.
- Step 3: Calculate centimeters = meters * 100.0.
- Step 4: Calculate feet = km * 3280.84.
- Step 5: Calculate inches = feet * 12.0.
- Step 6: Display all conversion units clearly.

📄 SOURCE CODE IMPLEMENTATION (BCSE-001)

VERIFIED & TESTED

```
#include <stdio.h>

int main() {
    float km, meters, feet, inches, cm;

    printf("*** BCSE-001: Practical 2 - Distance Conversion ***\n");
    printf("Enter distance between two cities (in Kilometers): ");
    scanf("%f", &km);

    meters = km * 1000.0f;
    cm = meters * 100.0f;
    feet = km * 3280.84f;
    inches = feet * 12.0f;

    printf("\n--- Converted Units ---\n");
    printf("Distance in Meters      : %.2f m\n", meters);
    printf("Distance in Centimeters : %.2f cm\n", cm);
    printf("Distance in Feet        : %.2f ft\n", feet);
    printf("Distance in Inches      : %.2f in\n", inches);

    return 0;
}
```

📄 SAMPLE INPUT:

Distance in KM: 5.5

📄 EXPECTED OUTPUT:

Meters: 5500.00 m | Centimeters: 550000.00 cm | Feet:
18044.62 ft | Inches: 216535.44 in

Bitwise Operators Demonstration (&, |, ^, ~, <<, >>)

🎯 AIM / OBJECTIVE:

Write a program to demonstrate the use of bitwise operators in C: Bitwise AND, OR, XOR, NOT, Left Shift, and Right Shift with binary representations.

⚙️ ALGORITHM / STEP-BY-STEP PROCEDURE:

- Step 1: Declare unsigned variables a and b.
- Step 2: Prompt user for two 8-bit integers (0 to 255).
- Step 3: Evaluate a & b, a | b, a ^ b, ~a, a << 2, b >> 1.
- Step 4: Output both integer and 8-bit binary representations for each.

📄 SOURCE CODE IMPLEMENTATION (BCSE-001)

VERIFIED & TESTED

```
#include <stdio.h>

void printBinary(unsigned int n) {
    for (int i = 7; i >= 0; i--) {
        printf("%d", (n >> i) & 1);
    }
}

int main() {
    unsigned char a, b;
    printf("*** BCSE-001: Practical 3 - Bitwise Operators ***\n");
    printf("Enter two integer values (0-255): ");
    scanf("%hhu %hhu", &a, &b);

    printf("a = %3d [", a); printBinary(a); printf("\n");
    printf("b = %3d [", b); printBinary(b); printf("\n");

    printf("Bitwise AND (a & b) : %3d [", a & b); printBinary(a & b); printf("\n");
    printf("Bitwise OR (a | b) : %3d [", a | b); printBinary(a | b); printf("\n");
    printf("Bitwise XOR (a ^ b) : %3d [", a ^ b); printBinary(a ^ b); printf("\n");
    printf("Bitwise NOT (~a) : %3d [", (unsigned char)~a); printBinary(~a); printf("\n");
    printf("Left Shift (a << 2) : %3d [", a << 2); printBinary(a << 2); printf("\n");
    printf("Right Shift (b >> 1) : %3d [", b >> 1); printBinary(b >> 1); printf("\n");

    return 0;
}
```

📄 SAMPLE INPUT:

Values: 12 25

📄 EXPECTED OUTPUT:

a & b = 8 | a | b = 29 | a ^ b = 21 | ~a = 243 | a << 2 = 48 | b >> 1 = 12

Largest of Three Numbers using Conditional (Ternary) Operator

AIM / OBJECTIVE:

Write a C program to find the largest of three given numbers using the conditional/ternary operator (? :).

ALGORITHM / STEP-BY-STEP PROCEDURE:

- Step 1: Input three numbers a, b, and c.
- Step 2: Use nested ternary expression: `max = (a > b) ? ((a > c) ? a : c) : ((b > c) ? b : c);`
- Step 3: Print the evaluated largest number.

SOURCE CODE IMPLEMENTATION (BCSE-001)

VERIFIED & TESTED

```
#include <stdio.h>

int main() {
    int a, b, c, largest;
    printf("*** BCSE-001: Practical 4 - Largest using Ternary Operator ***\n");
    printf("Enter three numbers (a, b, c): ");
    scanf("%d %d %d", &a, &b, &c);

    largest = (a > b) ? ((a > c) ? a : c) : ((b > c) ? b : c);

    printf("The largest number among %d, %d, and %d is: %d\n", a, b, c, largest);
    return 0;
}
```

SAMPLE INPUT:

Numbers: 45 89 23

EXPECTED OUTPUT:

The largest number among 45, 89, and 23 is: 89

Leap Year Verification using Boolean Logic

AIM / OBJECTIVE:

Write a C program to determine whether a given year is a leap year or not using conditional expressions.

ALGORITHM / STEP-BY-STEP PROCEDURE:

- Step 1: Read year as integer.
- Step 2: Test condition: $(\text{year} \% 400 == 0) \parallel (\text{year} \% 4 == 0 \ \&\& \ \text{year} \% 100 != 0)$.
- Step 3: If true, print Leap Year (366 days).
- Step 4: Else, print Not a Leap Year (365 days).

SOURCE CODE IMPLEMENTATION (BCSE-001)

VERIFIED & TESTED

```
#include <stdio.h>

int main() {
    int year;
    printf("*** BCSE-001: Practical 5 - Leap Year Checker ***");
    printf("\n");
    printf("Enter a year: ");
    scanf("%d", &year);

    if ((year % 400 == 0) || (year % 4 == 0 && year % 100 != 0)) {
        printf("Year %d is a LEAP YEAR (366 days).", year);
    } else {
        printf("Year %d is NOT a leap year (365 days).", year);
    }
    return 0;
}
```

SAMPLE INPUT:

Year: 2024

EXPECTED OUTPUT:

Year 2024 is a LEAP YEAR (366 days).

Student Grade & Division Evaluator for 5 Subjects

AIM / OBJECTIVE:

Write a C program that takes marks of 5 subjects, calculates total marks, percentage, and assigns grades according to university marks brackets.

ALGORITHM / STEP-BY-STEP PROCEDURE:

- Step 1: Input marks for 5 subjects (out of 100).
- Step 2: Validate all marks in range [0, 100].
- Step 3: Total = m1 + m2 + m3 + m4 + m5, Percentage = Total / 5.0.
- Step 4: Assign Grade based on university score brackets.
- Step 5: Output summary.

SOURCE CODE IMPLEMENTATION (BCSE-001)

VERIFIED & TESTED

```
#include <stdio.h>

int main() {
    float m1, m2, m3, m4, m5, total, percentage;

    printf("*** BCSE-001: Practical 6 - Grade Calculator ***\n");
    printf("Enter marks for 5 subjects (out of 100):\n");
    scanf("%f %f %f %f %f", &m1, &m2, &m3, &m4, &m5);

    total = m1 + m2 + m3 + m4 + m5;
    percentage = total / 5.0f;

    printf("Total Marks: %.2f / 500\n", total);
    printf("Percentage : %.2f%%\n", percentage);
    printf("Result      : ");

    if (percentage >= 90.0) printf("Grade O (Outstanding)\n");
    else if (percentage >= 80.0) printf("Grade A+ (Excellent)\n");
    else if (percentage >= 70.0) printf("Grade A (Very Good)\n");
    else if (percentage >= 60.0) printf("Grade B+ (Good)\n");
    else if (percentage >= 50.0) printf("Grade B (Above Average)\n");
    else if (percentage >= 40.0) printf("Grade C (Pass)\n");
    else printf("Grade F (FAIL / Re-appear)\n");

    return 0;
}
```

SAMPLE INPUT:

Marks: 85 92 78 88 90

EXPECTED OUTPUT:

Total: 433.00/500 | Percentage: 86.60% | Result: Grade A+ (Excellent)

Prefix vs Postfix Increment and Decrement Operator Analysis

AIM / OBJECTIVE:

Write a program to demonstrate the difference between prefix and postfix increment (++x, x++) and decrement (--x, x--) operators.

ALGORITHM / STEP-BY-STEP PROCEDURE:

- Step 1: Declare integer a = 10.
- Step 2: Compare b = a++ vs b = ++a.
- Step 3: Compare b = a-- vs b = --a.
- Step 4: Print the outputs.

SOURCE CODE IMPLEMENTATION (BCSE-001)

VERIFIED & TESTED

```
#include <stdio.h>

int main() {
    int a = 10, b;

    printf("*** BCSE-001: Practical 7 - Prefix vs Postfix Operators ***
");
    printf("Initial value of a = %d
", a);

    b = a++;
    printf("Postfix (b = a++): b = %d, a becomes = %d
", b, a);

    b = ++a;
    printf("Prefix (b = ++a): b = %d, a becomes = %d
", b, a);

    b = a--;
    printf("Postfix (b = a--): b = %d, a becomes = %d
", b, a);

    b = --a;
    printf("Prefix (b = --a): b = %d, a becomes = %d
", b, a);

    return 0;
}
```

SAMPLE INPUT:

Initial a = 10

EXPECTED OUTPUT:

Postfix a++: b=10, a=11 | Prefix ++a: b=12, a=12

Menu-Driven Switch Program (Even/Odd, Pos/Neg, Square, Square Root)

🎯 AIM / OBJECTIVE:

Write a menu-driven program in C using switch case to perform: 1. Check Even/Odd, 2. Check Positive/Negative, 3. Calculate Square, 4. Calculate Square Root.

⚙️ ALGORITHM / STEP-BY-STEP PROCEDURE:

- Step 1: Display interactive menu options 1 to 4.
- Step 2: Read user choice.
- Step 3: In switch-case, execute corresponding calculation.
- Step 4: Handle invalid choices with default case.

📄 SOURCE CODE IMPLEMENTATION (BCSE-001)

VERIFIED & TESTED

```
#include <stdio.h>
#include <math.h>

int main() {
    int choice;
    double num;

    printf("*** BCSE-001: Practical 8 - Menu-Driven Switch System ***\n");
    printf("1. Check Even or Odd\n");
    printf("2. Check Positive, Negative or Zero\n");
    printf("3. Find Square\n");
    printf("4. Find Square Root\n");
    printf("Enter your choice (1-4): ");
    scanf("%d", &choice);

    printf("Enter number: ");
    scanf("%lf", &num);

    switch (choice) {
        case 1:
            if ((int)num % 2 == 0)
                printf("%d is EVEN.\n", (int)num);
            else
                printf("%d is ODD.\n", (int)num);
            break;
        case 2:
            if (num > 0) printf("%.2lf is POSITIVE.\n", num);
            else if (num < 0) printf("%.2lf is NEGATIVE.\n", num);
            else printf("The number is ZERO.\n");
            break;
        case 3:
            printf("Square of %.2lf is: %.2lf\n", num, num * num);
            break;
        case 4:
            if (num >= 0)
                printf("Square root of %.2lf is: %.4lf\n", num, sqrt(num));
            else
                printf("Error: Cannot compute real square root of negative number!\n");
    }
}
```

```
        break;
    default:
        printf("Invalid choice selected!
");
    }
    return 0;
}
```

**SAMPLE INPUT:**

Choice: 4, Number: 49

**EXPECTED OUTPUT:**

Square root of 49.00 is: 7.0000

Sum of All Integers Between 100 and 200 Divisible by 5

🎯 AIM / OBJECTIVE:

Write a program in C to find the sum of all integers greater than 100 and less than 200 that are completely divisible by 5.

⚙️ ALGORITHM / STEP-BY-STEP PROCEDURE:

- Step 1: Set sum = 0 and count = 0.
- Step 2: Loop i from 101 to 199.
- Step 3: If $i \% 5 == 0$, add i to sum and print i.
- Step 4: Output total count and sum.

📄 SOURCE CODE IMPLEMENTATION (BCSE-001)

VERIFIED & TESTED

```
#include <stdio.h>

int main() {
    int sum = 0, count = 0;

    printf("*** BCSE-001: Practical 9 - Sum of Integers Divisible by 5 (100 to 200) ***");
    printf("\n");
    printf("Numbers divisible by 5 between 100 and 200:");
    printf("\n");

    for (int i = 101; i < 200; i++) {
        if (i % 5 == 0) {
            printf("%d ", i);
            sum += i;
            count++;
        }
    }

    printf("\n");

    printf("Total count : %d\n");
    printf("Sum of numbers: %d\n", count, sum);
    return 0;
}
```

📄 SAMPLE INPUT:

Fixed bounds (101 to 199)

📄 EXPECTED OUTPUT:

105 110 115 120 125 130 135 140 145 150 155 160 165
170 175 180 185 190 195 | Sum = 2850

Armstrong Numbers Discovery in Range [m, n]

AIM / OBJECTIVE:

Write a C program to find and display all Armstrong numbers within a given range [m, n].

ALGORITHM / STEP-BY-STEP PROCEDURE:

- Step 1: Input range [m, n].
- Step 2: For each number from m to n, count digits (d).
- Step 3: Sum each digit raised to power d.
- Step 4: If sum == num, print num as Armstrong number.

SOURCE CODE IMPLEMENTATION (BCSE-001)

VERIFIED & TESTED

```
#include <stdio.h>
#include <math.h>

int isArmstrong(int num) {
    int temp = num, digits = 0, sum = 0;
    while (temp > 0) {
        digits++;
        temp /= 10;
    }
    temp = num;
    while (temp > 0) {
        int rem = temp % 10;
        sum += (int)round(pow(rem, digits));
        temp /= 10;
    }
    return sum == num;
}

int main() {
    int m, n, found = 0;
    printf("*** BCSE-001: Practical 10 - Armstrong Numbers in Range [m, n] ***
");
    printf("Enter range [m, n]: ");
    scanf("%d %d", &m, &n);

    printf("Armstrong numbers between %d and %d are:
", m, n);
    for (int i = m; i <= n; i++) {
        if (isArmstrong(i)) {
            printf("%d ", i);
            found++;
        }
    }
    if (!found) printf("None found in this range.");
    printf("
Total found: %d
", found);

    return 0;
}
```

SAMPLE INPUT:

Range: 1 1000

EXPECTED OUTPUT:

Armstrong numbers: 1 2 3 4 5 6 7 8 9 153 370 371 407 |
Total found: 13

Linear Search & Binary Search on 1D Array

AIM / OBJECTIVE:

Write a C program to implement both Linear Search and Binary Search on a 1D array, measuring comparison counts and displaying element positions.

ALGORITHM / STEP-BY-STEP PROCEDURE:

- Step 1: Read array size n and sorted elements.
- Step 2: Linear search scans index 0 to n-1 sequentially.
- Step 3: Binary search compares target with mid, adjusting low and high bounds.
- Step 4: Output index and position.

SOURCE CODE IMPLEMENTATION (BCSE-001)

VERIFIED & TESTED

```
#include <stdio.h>

int linearSearch(int arr[], int n, int target) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == target) return i;
    }
    return -1;
}

int binarySearch(int arr[], int n, int target) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

int main() {
    int n, target, arr[50];
    printf("*** BCSE-001: Practical 11 - Linear & Binary Search ***\n");
    printf("Enter size of sorted array: ");
    scanf("%d", &n);

    printf("Enter %d sorted elements: ", n);
    for (int i = 0; i < n; i++) scanf("%d", &arr[i]);

    printf("Enter value to search: ");
    scanf("%d", &target);

    int linPos = linearSearch(arr, n, target);
    int binPos = binarySearch(arr, n, target);

    if (linPos != -1) {
        printf("Linear Search: Element found at index %d (Position %d)\n", linPos, linPos + 1);
        printf("Binary Search: Element found at index %d (Position %d)\n", binPos, binPos + 1);
    } else {
        printf("Element %d not found in the array.\n", target);
    }
    return 0;
}
```

SAMPLE INPUT:

EXPECTED OUTPUT:

Size: 6 | Elements: 11 22 33 44 55 66 | Search: 44

Found at index 3 (Position 4)

Matrix Operations (Addition, Subtraction, Multiplication & Transpose)

🎯 AIM / OBJECTIVE:

Write a menu-driven program in C to perform Matrix operations: Matrix Addition, Matrix Subtraction, Matrix Multiplication, and Matrix Transpose using 2D arrays.

⚙️ ALGORITHM / STEP-BY-STEP PROCEDURE:

- Step 1: Input dimensions (r1, c1) and (r2, c2) for matrices A and B.
- Step 2: Check dimension constraints for Add/Sub and Multiply.
- Step 3: Execute chosen operation using nested loops.
- Step 4: Output formatted 2D matrix.

📄 SOURCE CODE IMPLEMENTATION (BCSE-001)

VERIFIED & TESTED

```
#include <stdio.h>

void readMatrix(int r, int c, int mat[10][10], char name) {
    printf("Enter elements for Matrix %c (%dx%d):", name, r, c);
    for (int i = 0; i < r; i++)
        for (int j = 0; j < c; j++)
            scanf("%d", &mat[i][j]);
}

void printMatrix(int r, int c, int mat[10][10]) {
    for (int i = 0; i < r; i++) {
        for (int j = 0; j < c; j++) printf("%4d ", mat[i][j]);
        printf("\n");
    }
}

int main() {
    int A[10][10], B[10][10], C[10][10], T[10][10];
    int r1, c1, r2, c2, choice;

    printf("*** BCSE-001: Practical 12 - 2D Matrix Suite ***\n");
    printf("Enter rows and cols of Matrix A: ");
    scanf("%d %d", &r1, &c1);
    readMatrix(r1, c1, A, 'A');

    printf("Enter rows and cols of Matrix B: ");
    scanf("%d %d", &r2, &c2);
    readMatrix(r2, c2, B, 'B');

    printf("Select: 1. Add  2. Subtract  3. Multiply  4. Transpose A\n");
    printf("Choice: ");
    scanf("%d", &choice);

    switch (choice) {
        case 1:
            if (r1 == r2 && c1 == c2) {
                for (int i = 0; i < r1; i++)
                    for (int j = 0; j < c1; j++) C[i][j] = A[i][j] + B[i][j];
                printf("Result (A + B):\n");
                printMatrix(r1, c1, C);
            } else printf("Error: Dimension mismatch!\n");
            break;
        case 2:
            if (r1 == r2 && c1 == c2) {
```

```

        for (int i = 0; i < r1; i++)
            for (int j = 0; j < c1; j++) C[i][j] = A[i][j] - B[i][j];
        printf("
Result (A - B):
");
        printMatrix(r1, c1, C);
    } else printf("Error: Dimension mismatch!
");
    break;
case 3:
    if (c1 == r2) {
        for (int i = 0; i < r1; i++) {
            for (int j = 0; j < c2; j++) {
                C[i][j] = 0;
                for (int k = 0; k < c1; k++) C[i][j] += A[i][k] * B[k][j];
            }
        }
        printf("
Result (A * B):
");
        printMatrix(r1, c2, C);
    } else printf("Error: Multiplication dimension mismatch!
");
    break;
case 4:
    for (int i = 0; i < r1; i++)
        for (int j = 0; j < c1; j++) T[j][i] = A[i][j];
    printf("
Transpose of A (%dx%d):
", c1, r1);
    printMatrix(c1, r1, T);
    break;
default: printf("Invalid choice!
");
}
return 0;
}

```



SAMPLE INPUT:

Choice: 3 (Multiplication) on 2x2 matrices



EXPECTED OUTPUT:

Computed product matrix printed.

Built-in String Library Functions Demonstration (string.h)

🎯 AIM / OBJECTIVE:

Write a C program to demonstrate the use of built-in string functions: strlen(), strcpy(), strcat(), strcmp(), and strrev().

⚙️ ALGORITHM / STEP-BY-STEP PROCEDURE:

- Step 1: Declare character arrays s1 and s2.
- Step 2: Use strlen() to measure lengths.
- Step 3: Use strcpy() to copy s1 to buffer.
- Step 4: Use strcmp() to compare s1 and s2.
- Step 5: Use strcat() to concatenate s2 to s1.

📄 SOURCE CODE IMPLEMENTATION (BCSE-001)

VERIFIED & TESTED

```
#include <stdio.h>
#include <string.h>

int main() {
    char s1[100] = "Hello";
    char s2[100] = "World";
    char copy[100];

    printf("*** BCSE-001: Practical 13 - Built-in String Functions ***
");
    printf("Initial s1: %s (Length: %zu)
", s1, strlen(s1));
    printf("Initial s2: %s (Length: %zu)
", s2, strlen(s2));

    strcpy(copy, s1);
    printf("
1. strcpy: Copied s1 -> '%s'
", copy);

    int cmp = strcmp(s1, s2);
    printf("2. strcmp: Result: %d (%s)
", s1, s2, cmp == 0 ? "Equal" : (cmp < 0 ? "s1 < s2" : "s1 > s2"));

    strcat(s1, " ");
    strcat(s1, s2);
    printf("3. strcat: Concatenated -> '%s' (Length: %zu)
", s1, strlen(s1));

    return 0;
}
```

📄 SAMPLE INPUT:

Strings: 'Hello' and 'World'

📄 EXPECTED OUTPUT:

Concatenated: 'Hello World' (Length: 11)

User-Defined Implementations of String Functions (Without string.h)

🎯 AIM / OBJECTIVE:

Write a C program to implement custom user-defined functions for finding length, copying, concatenating, comparing, and reversing strings without including <string.h>.

⚙️ ALGORITHM / STEP-BY-STEP PROCEDURE:

- Step 1: Implement my_strlen() using loop.
- Step 2: Implement my_strcpy() char-by-char.
- Step 3: Implement my_strcat() by seeking null terminator.
- Step 4: Implement my_strcmp() by comparing ASCII values.
- Step 5: Implement my_strrev() by swapping from both ends.

📄 SOURCE CODE IMPLEMENTATION (BCSE-001)

VERIFIED & TESTED

```
#include <stdio.h>

int my_strlen(const char *str) {
    int len = 0;
    while (str[len] != '\0') len++;
    return len;
}

void my_strcpy(char *dest, const char *src) {
    int i = 0;
    while (src[i] != '\0') {
        dest[i] = src[i];
        i++;
    }
    dest[i] = '\0';
}

void my_strcat(char *dest, const char *src) {
    int i = my_strlen(dest);
    int j = 0;
    while (src[j] != '\0') {
        dest[i + j] = src[j];
        j++;
    }
    dest[i + j] = '\0';
}

int my_strcmp(const char *s1, const char *s2) {
    int i = 0;
    while (s1[i] != '\0' && s2[i] != '\0') {
        if (s1[i] != s2[i]) return s1[i] - s2[i];
        i++;
    }
    return s1[i] - s2[i];
}

void my_strrev(char *str) {
    int i = 0, j = my_strlen(str) - 1;
    while (i < j) {
        char temp = str[i];
        str[i] = str[j];
        str[j] = temp;
        i++;
        j--;
    }
}

int main() {
    char str1[100] = "Antigravity";
    char str2[100] = "Engine";
```

```

char buf[100];

printf("*** BCSE-001: Practical 14 - Custom String Functions ***
");
printf("Custom Length of '%s': %d
", str1, my_strlen(str1));

my_strcpy(buf, str1);
printf("Custom Strcpy: Copied -> '%s'
", buf);

my_strcat(buf, str2);
printf("Custom Strcat: Result -> '%s'
", buf);

my_strrev(buf);
printf("Custom Strrev: Reversed -> '%s'
", buf);

return 0;
}

```

SAMPLE INPUT:

Strings: 'Antigravity' and 'Engine'

EXPECTED OUTPUT:

Length: 11 | Concatenated: AntigravityEngine |
Reversed: enignEytivarginA

Call by Value vs Call by Reference (Swapping Two Variables)

AIM / OBJECTIVE:

Write a program in C to swap two numbers using: 1. Call by Value, 2. Call by Reference (pointers), explaining differences in caller memory.

ALGORITHM / STEP-BY-STEP PROCEDURE:

- Step 1: Declare integers a = 10, b = 20.
- Step 2: In swapByValue(a, b), values are copied into function local variables; original variables unchanged.
- Step 3: In swapByReference(&a, &b), addresses are passed and dereferenced; original variables are swapped.
- Step 4: Output states.

SOURCE CODE IMPLEMENTATION (BCSE-001)

VERIFIED & TESTED

```
#include <stdio.h>

void swapByValue(int x, int y) {
    int temp = x;
    x = y;
    y = temp;
    printf("[Inside swapByValue] x = %d, y = %d\n", x, y);
}

void swapByReference(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
    printf("[Inside swapByReference] *x = %d, *y = %d\n", *x, *y);
}

int main() {
    int a = 10, b = 20;

    printf("*** BCSE-001: Practical 15 - Call by Value vs Reference ***\n");
    printf("Original: a = %d, b = %d\n", a, b);

    printf("--- Testing Call by Value ---\n");
    swapByValue(a, b);
    printf("In main after swapByValue: a = %d, b = %d (UNMODIFIED)\n", a, b);

    printf("--- Testing Call by Reference ---\n");
    swapByReference(&a, &b);
    printf("In main after swapByReference: a = %d, b = %d (SUCCESSFULLY SWAPPED)\n", a, b);

    return 0;
}
```

SAMPLE INPUT:

Initial a=10, b=20

EXPECTED OUTPUT:

After Call by Value: a=10, b=20 | After Call by Reference: a=20, b=10

Recursive Computation of Factorial

AIM / OBJECTIVE:

Write a program in C to calculate the factorial of an integer using recursion with stack base case.

ALGORITHM / STEP-BY-STEP PROCEDURE:

- Step 1: Define factorial(n).
- Step 2: Base case: if $n \leq 1$, return 1.
- Step 3: Recursive case: return $n * \text{factorial}(n - 1)$.
- Step 4: Print factorial value.

SOURCE CODE IMPLEMENTATION (BCSE-001)

VERIFIED & TESTED

```
#include <stdio.h>

unsigned long long factorial(int n) {
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}

int main() {
    int num;
    printf("*** BCSE-001: Practical 16 - Recursive Factorial ***\n");
    printf("Enter a positive integer: ");
    scanf("%d", &num);

    if (num < 0) {
        printf("Error: Factorial of negative number does not exist!\n");
    } else {
        printf("Factorial of %d (%d!) = %llu\n", num, num, factorial(num));
    }
    return 0;
}
```

SAMPLE INPUT:

Input: 6

EXPECTED OUTPUT:

Factorial of 6 (6!) = 720

Array Reversal & Memory Address Inspection via Pointers

AIM / OBJECTIVE:

Write a C program to read an array of integers and print the elements in reverse order along with their physical hexadecimal memory addresses using pointer arithmetic.

ALGORITHM / STEP-BY-STEP PROCEDURE:

- Step 1: Read n elements into array arr.
- Step 2: Initialize pointer ptr to arr + n - 1.
- Step 3: Loop backward printing *ptr and memory address (void*)ptr.
- Step 4: Decrement pointer.

SOURCE CODE IMPLEMENTATION (BCSE-001)

VERIFIED & TESTED

```
#include <stdio.h>

int main() {
    int n, arr[50];
    printf("*** BCSE-001: Practical 17 - Array & Pointer Traversal ***
");
    printf("Enter number of elements: ");
    scanf("%d", &n);

    printf("Enter %d elements: ", n);
    for (int i = 0; i < n; i++) scanf("%d", &arr[i]);

    int *ptr = arr + n - 1;

    printf("
Elements in Reverse Order with Memory Addresses:
");
    printf("-----
");
    printf("Index | Value | Memory Address
");
    printf("-----
");
    for (int i = n - 1; i >= 0; i--) {
        printf("%5d | %5d | %p
", i, *ptr, (void *)ptr);
        ptr--;
    }
    return 0;
}
```

SAMPLE INPUT:

Size: 5 | Elements: 10 20 30 40 50

EXPECTED OUTPUT:

Reversed array printed with contiguous 4-byte memory addresses.

Function Returning Pointer to Larger of Two Integers

AIM / OBJECTIVE:

Write a C program with a user-defined function that accepts two pointer arguments and returns a pointer to the larger integer value.

ALGORITHM / STEP-BY-STEP PROCEDURE:

- Step 1: Define `int* findLarger(int *x, int *y)`.
- Step 2: Compare `*x` and `*y`, return `x` if `*x >= *y`, else return `y`.
- Step 3: In `main()`, pass `&a` and `&b` to `findLarger()`.
- Step 4: Dereference returned pointer to access max value.

SOURCE CODE IMPLEMENTATION (BCSE-001)

VERIFIED & TESTED

```
#include <stdio.h>

int* findLarger(int *x, int *y) {
    if (*x >= *y) return x;
    return y;
}

int main() {
    int a, b;
    printf("*** BCSE-001: Practical 18 - Function Returning Pointer ***\n");
    printf("Enter two integer values: ");
    scanf("%d %d", &a, &b);

    int *maxPtr = findLarger(&a, &b);

    printf("Value of a: %d at %p\n", a, (void *)&a);
    printf("Value of b: %d at %p\n", b, (void *)&b);
    printf("The larger value is: %d (Stored at Address: %p)\n", *maxPtr, (void *)maxPtr);

    return 0;
}
```

SAMPLE INPUT:

Values: 72 45

EXPECTED OUTPUT:

The larger value is 72 (Stored at address of a)

Structure personal (Name, Date of Joining, Salary) Database

🎯 AIM / OBJECTIVE:

Write a program in C to declare a structure personal that includes Person Name, Date of Joining (nested structure), and Salary. Read and display records of multiple persons.

⚙️ ALGORITHM / STEP-BY-STEP PROCEDURE:

- Step 1: Define struct Date { int day, month, year; }.
- Step 2: Define struct Personal { char name[50]; struct Date doj; float salary; }.
- Step 3: Read array of n personal structures.
- Step 4: Display tabular formatted report.

📄 SOURCE CODE IMPLEMENTATION (BCSE-001)

VERIFIED & TESTED

```
#include <stdio.h>

struct Date {
    int day;
    int month;
    int year;
};

struct Personal {
    char name[50];
    struct Date doj;
    float salary;
};

int main() {
    int n;
    struct Personal staff[10];

    printf("*** BCSE-001: Practical 19 - Structure Personal Database ***\n");
    printf("Enter number of persons (max 10): ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("
--- Person %d ---
", i + 1);
        printf("Enter Name: ");
        scanf("%[^
]", staff[i].name);
        printf("Enter Date of Joining (DD MM YYYY): ");
        scanf("%d %d %d", &staff[i].doj.day, &staff[i].doj.month, &staff[i].doj.year);
        printf("Enter Monthly Salary (INR): ");
        scanf("%f", &staff[i].salary);
    }

    printf("
===== PERSONAL RECORDS =====
");
    printf("%-20s | %-12s | %-12s
", "Name", "Joining Date", "Salary");
    printf("-----
");
    for (int i = 0; i < n; i++) {
        printf("%-20s | %02d/%02d/%04d | Rs. %10.2f
",
            staff[i].name,
            staff[i].doj.day, staff[i].doj.month, staff[i].doj.year,
            staff[i].salary);
    }
}
```

```
return 0;  
}
```

**SAMPLE INPUT:**

Name: Rahul Sharma, DOJ: 15 08 2023, Salary: 65000

**EXPECTED OUTPUT:**

Formatted tabular employee record.

Dynamic Memory Allocation (malloc, realloc & free) for Strings

🎯 AIM / OBJECTIVE:

Write a C program to dynamically allocate memory for a character string using malloc(), read text, resize the memory block using realloc(), append text, and free memory.

⚙️ ALGORITHM / STEP-BY-STEP PROCEDURE:

- Step 1: Allocate initial block with malloc().
- Step 2: Check for NULL allocation failure.
- Step 3: Copy initial string into allocated block.
- Step 4: Resize block using realloc() and append string.
- Step 5: Free memory using free().

📄 SOURCE CODE IMPLEMENTATION (BCSE-001)

VERIFIED & TESTED

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main() {
    char *str;
    int initialSize = 15;
    int expandedSize = 40;

    printf("*** BCSE-001: Practical 20 - Dynamic Memory Allocation ***
");

    str = (char *)malloc(initialSize * sizeof(char));
    if (str == NULL) {
        printf("Memory allocation failed!
");
        return 1;
    }
    strcpy(str, "Hello MMEC");
    printf("1. Memory allocated with malloc(%d bytes): '%s'
", initialSize, str);

    str = (char *)realloc(str, expandedSize * sizeof(char));
    if (str == NULL) {
        printf("Reallocation failed!
");
        return 1;
    }
    strcat(str, " - Computer Science");
    printf("2. Memory reallocated with realloc(%d bytes): '%s'
", expandedSize, str);

    free(str);
    str = NULL;
    printf("3. Memory successfully freed using free().
");

    return 0;
}
```

📄 SAMPLE INPUT:

Fixed test string 'Hello MMEC'

📄 EXPECTED OUTPUT:

Allocated -> Reallocated -> Freed safely.

File Handling: Store & Retrieve 15 Employee Records

🎯 AIM / OBJECTIVE:

Write a C program to store details (Name, EmpCode, ContactNo, Department) of 15 employees in a file using `fprintf()`, and then read & display records using `fscanf()`.

⚙️ ALGORITHM / STEP-BY-STEP PROCEDURE:

- Step 1: Open 'employees.txt' in write mode 'w'.
- Step 2: Write employee records using `fprintf()`. Close file.
- Step 3: Open file in read mode 'r'.
- Step 4: Read records using `fscanf()` and display formatted table.
- Step 5: Close file.

📄 SOURCE CODE IMPLEMENTATION (BCSE-001)

VERIFIED & TESTED

```
#include <stdio.h>
#include <stdlib.h>

struct Employee {
    char name[50];
    int empCode;
    char phone[15];
    char dept[30];
};

int main() {
    FILE *fp;
    int n = 3;
    struct Employee emp;

    printf("*** BCSE-001: Practical 21 - Employee File Database ***\n");

    fp = fopen("employees.txt", "w");
    if (fp == NULL) {
        printf("Error opening file for writing!\n");
        return 1;
    }

    printf("Enter details for %d employees:\n", n);
    for (int i = 0; i < n; i++) {
        printf("Employee # %d\n", i + 1);
        printf("Enter EmpCode: ");
        scanf("%d", &emp.empCode);
        printf("Enter Name (Single word): ");
        scanf("%s", emp.name);
        printf("Enter Phone: ");
        scanf("%s", emp.phone);
        printf("Enter Department: ");
        scanf("%s", emp.dept);

        fprintf(fp, "%d %s %s %s\n", emp.empCode, emp.name, emp.phone, emp.dept);
    }
    fclose(fp);
    printf("[SUCCESS] Records saved to 'employees.txt'.\n");

    fp = fopen("employees.txt", "r");
    if (fp == NULL) {
```

```

        printf("Error opening file for reading!
");
        return 1;
    }

    printf("
===== READ BACK FROM FILE =====
");
    printf("%-8s | %-15s | %-12s | %-10s
", "Code", "Name", "Phone", "Dept");
    printf("-----
");
    while (fscanf(fp, "%d %s %s %s", &emp.empCode, emp.name, emp.phone, emp.dept) == 4) {
        printf("%-8d | %-15s | %-12s | %-10s
", emp.empCode, emp.name, emp.phone, emp.dept);
    }
    fclose(fp);

    return 0;
}

```



SAMPLE INPUT:

Code: 101, Name: Amit, Phone: 9876543210, Dept: CSE



EXPECTED OUTPUT:

File written and read back into formatted console table.



Project Description: Full real-time Pacman game implementing 2D grid matrix mapping, live keyboard input reading (conio.h / Windows console API), dynamic score tracking, ghost movements, and collision detection.

Core Engineering Modules:

- ✓ Interactive 2D ASCII game grid with boundary walls and score dots
- ✓ Player movement using W/A/S/D or Arrow keys without lagging screen refresh
- ✓ Automated wandering ghost algorithm with collision check
- ✓ Real-time live score board & life counter
- ✓ Game over & High-score state management



CAPSTONE ENGINE SOURCE CODE

ACADEMIC SUBMISSION READY

```
#include <stdio.h>
#include <stdlib.h>
#include <conio.h>
#include <windows.h>

#define H 15
#define W 40

char board[H][W];
int pac_x = 1, pac_y = 1;
int ghost_x = 10, ghost_y = 20;
int score = 0, lives = 3, food = 0;

void initBoard() {
    for (int i = 0; i < H; i++) {
        for (int j = 0; j < W; j++) {
            if (i == 0 || i == H - 1 || j == 0 || j == W - 1) {
                board[i][j] = '#';
            } else if ((i == 3 && j > 5 && j < 35) || (i == 10 && j > 5 && j < 35)) {
                board[i][j] = '#';
            } else {
                board[i][j] = '.';
                food++;
            }
        }
    }
    board[pac_x][pac_y] = 'C';
    board[ghost_x][ghost_y] = 'X';
}

void draw() {
    COORD coord = {0, 0};
    SetConsoleCursorPosition(GetStdHandle(STD_OUTPUT_HANDLE), coord);
    printf("=== GOD-LEVEL PACMAN IN C (BCSE-001 CAPSTONE) ===");
}
```

Q1. What are tokens in C language and what are their types?

Answer: A token is the smallest individual unit in a C program. The 6 types are: Keywords (32 standard keywords like int, if), Identifiers (names given to variables/functions), Constants (fixed values like 100, 3.14), String Literals ("Hello"), Operators (+, -, *), and Special Characters/Punctuators (;, {}, ()).

Q2. What is the difference between = and == in C?

Answer: = is the assignment operator used to assign the right-hand value to a left-hand variable (e.g., a = 5). == is the relational equality operator used to compare two values for equality, evaluating to 1 (true) or 0 (false).

Q3. What is the significance of the void main() vs int main() in C?

Answer: According to C89/C99/C11 standards, main() must return an integer to the Operating System to signify exit status. Returning 0 indicates successful execution, while non-zero indicates an error code. void main() is non-standard and rejected by modern ISO C compilers.

Q4. Explain the difference between prefix (++x) and postfix (x++) increment operators.

Answer: Prefix (++x) follows the rule 'increment first, then use'. It increments the variable value before evaluating the expression. Postfix (x++) follows 'use first, then increment'. It evaluates the current value in the expression first and increments the variable immediately afterward.

Q5. What is a dangling pointer and how can it be avoided?

Answer: A dangling pointer is a pointer that points to a memory location that has already been deallocated/freed. It can lead to segmentation faults or undefined behavior. It is avoided by assigning NULL to the pointer immediately after free() (e.g., free(ptr); ptr = NULL;).

Q6. What is a memory leak and how do you prevent it in C?

Answer: A memory leak occurs when dynamically allocated heap memory (via malloc, calloc, realloc) is no longer needed but has not been freed using free(). The memory remains marked as occupied until program termination. Always pair every malloc() with a free().

Q7. What is the difference between malloc() and calloc()?

Answer: malloc(size) takes a single parameter representing total bytes and leaves allocated memory uninitialized (containing garbage values). calloc(num, size) takes two parameters (element count and size per element) and automatically initializes all allocated bytes to zero.

Q8. What is the difference between Structure and Union in C?

Answer: In a struct, each member gets its own separate memory location, so total size >= sum of sizes of all members. In a union, all members share the same memory location, and total size equals the size of its largest member. Only one member can be used at any given time in a union.

Q9. Explain call by value vs call by reference.

Answer: In call by value, copies of actual parameters are passed to formal parameters in the function stack frame. Changes made inside the function do NOT affect original variables. In call by reference, memory addresses (&) are passed as pointers (*). Modifying dereferenced values directly changes the caller variables.

Q10. What is recursion and what are its essential components?

Answer: Recursion is a programming technique where a function calls itself to solve smaller instances of the same problem. Essential components: 1. Base Case (stopping condition to avoid stack overflow), 2. Recursive Step (moving toward the base case), and 3. Return statement.

Q11. What is the role of the preprocessor in C?

Answer: The preprocessor runs before compilation. It handles directives starting with # such as #include (file inclusion), #define (macro definition and constant replacement), and conditional compilation (#ifdef, #ifndef, #endif).

Q12. What is the difference between a character array and a string literal in C?

Answer: A character array (`char arr[] = "Hi";`) is stored in the stack and its characters can be modified. A string literal (`char *ptr = "Hi";`) is stored in the read-only data segment; attempting to modify `*ptr` causes undefined behavior or a segmentation fault.

Q13. Explain bitwise operators (&, |, ^, ~, <<, >>) with use cases.

Answer: & (AND): mask bits; | (OR): set bits; ^ (XOR): toggle bits or swap without temp variable; ~ (NOT): invert bits; << (Left Shift): multiply by 2^n ; >> (Right Shift): divide by 2^n .

Q14. What are storage classes in C?

Answer: Storage classes define the scope, visibility, and lifetime of a variable. The four storage classes in C are: `auto` (local stack variable), `register` (stored in CPU register for speed), `static` (retains value across function calls), and `extern` (global variable shared across files).

Q15. How does binary search work and what is its prerequisite and time complexity?

Answer: Prerequisite: Array must be sorted. It compares target with mid element. If equal, target found; if smaller, searches left subarray; if larger, searches right subarray. Time complexity is $O(\log n)$ vs $O(n)$ in linear search.